MVRX Group
AI Email Consumption Specification: AECS-1
Version 1.0.0
3 July 2026

AECS-1: AI Email Consumption Specification

NormalizedEmail — A Community Standard for AI-Ready Email

Abstract

This specification defines a standardized structure — `NormalizedEmail` — for representing a parsed RFC 5322 email message in a form suitable for AI and LLM consumption, database storage, and conversation threading. It focuses exclusively on how a single message should be structured after normalization. Transport, authentication, mailbox management, and platform integration are out of scope.

Status of This Document

Version 1.0.0 — Final. This document is the normative definition of the AECS-1 standard. Machine-checkable artifacts (JSON Schema, conformance fixtures) are maintained alongside this specification in the `github.com/mvrxapp/mail` repository.

Comments, errata, and implementation experience reports may be sent to `specs@mvrx.app`. An errata page will be published at `https://mvrx.app/specs/aecs-1/errata` when the first erratum is recorded.

The reference implementation is `@mvrx/mail` (TypeScript). Conformance fixtures for the threading algorithm (§5) and timestamp rules (§6) are in `specs/conformance/fixtures/`.

Conventions Used in This Document

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **NOT RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals, as shown here.

Copyright Notice

1. Purpose

Raw email is noisy, inconsistently encoded, and poorly suited for direct consumption by AI systems or modern applications. A single message may contain quoted reply chains, HTML markup, MIME multipart boundaries, base64-encoded content, and legacy header formats.

AECS-1 defines a normalized representation that:

- Provides multiple content levels from raw to AI-optimized
- Preserves the original message when required
- Establishes a deterministic, stable threading model
- Remains flexible — no field beyond `messageId` and `threadId` is mandatory

---

1.1 Relation to Prior Art

The obvious prior art here is **JMAP** (RFC 8620 core protocol, RFC 8621 mail data model) — the IETF standard for structured, JSON-based email access. AECS-1 is not a replacement for it and deliberately does not compete on the same axis:

- **JMAP defines a transport and sync protocol** — how a client fetches, pushes, and incrementally syncs mailbox state over HTTP. AECS-1 defines none of that; it's transport-agnostic and only describes the shape of one already-fetched message. A JMAP client's `Email` object is a legitimate AECS-1 `parse()` input source, same as a raw RFC 5322 stream.
- **JMAP's `Email` object is a fidelity-preserving mirror of the message.** It doesn't define an AI/LLM-optimized content level — there's no equivalent of `content.clean` or `content.forAI` (quote-stripped, signature-stripped, bounded, prompt-injection-aware output). That gap is the entire reason AECS-1 exists.
- **Adopting JMAP is a much bigger commitment** — a full client/server sync protocol — than most AI-on-email use cases need. AECS-1 is scoped to the one narrow problem of "given a message, produce a normalized, AI-ready shape for it," so it can sit downstream of *any* transport: IMAP, JMAP, a raw inbound webhook, or Cloudflare Email Routing.

If you're building a full mail client, JMAP is very likely still the right protocol choice for sync — AECS-1 is complementary, not a competitor, at the normalization layer.

---

2. Core Principles

- **Flexible by design.** All fields except `messageId` and `threadId` are optional. Implementations populate what they can; unpopulated fields SHOULD be explicit `null` (consumers MUST accept omission too — §10).
- **Non-destructive.** The original raw message is preserved as an atomic field when included. Normalization layers are additions, not replacements.
- **Multiple content levels.** Consumers choose the level of processing that suits their use case — from raw RFC 5322 bytes to a clean, LLM-ready string.
- **Stable threading.** `threadId` is calculated deterministically from standard email headers. It must be identical for all messages in the same conversation, across implementations.
- **UTC everywhere.** All timestamps are Unix epoch integers (seconds). ISO 8601 strings, where provided, are always UTC.

---

3. NormalizedEmail Schema

```
{
  "messageId": "string",
  "threadId": "string",

  "metadata": {
    "from":    { "name": "string | null", "email": "string" },
    "to":      [{ "name": "string | null", "email": "string" }],
    "cc":      [{ "name": "string | null", "email": "string" }],
    "bcc":     [{ "name": "string | null", "email": "string" }],
    "subject": "string | null",
    "date":    "string | null",
    "timestamp": "number | null"
  },

  "content": {
    "rawFull": "string | null",
    "raw":     "string | null",
    "html":    "string | null",
    "text":    "string | null",
    "clean":   "string | null",
    "forAI":   "string | null"
  },

  "thread": {
    "position":  "number | null",
    "inReplyTo": "string | null",
    "references": ["string"]
  },

  "attachments": [
    {
      "id":          "string | null",
      "filename":    "string",
      "contentType": "string",
      "size":        "number",
      "cid":         "string | null"
    }
  ],

  "processing": {
    "processedAt": "string",
    "specVersion": "string"
  }
}
```

4. Field Definitions

4.1 Top-Level

Field	Type	Required	Description
`messageId`	string	**Yes**	The value of the `Message-ID` header, normalized (angle brackets stripped). When the header is absent or invalid (§5.1), implementations MUST assign a synthetic ID per §4.1.1. Unique per message.
`threadId`	string	**Yes**	Stable conversation identifier. Calculated deterministically — see Section 5.
`metadata`	object	No	Parsed header fields.
`content`	object	No	Message body at multiple processing levels.
`thread`	object	No	Threading position and header chain.
`attachments`	array	No	Metadata for each MIME attachment. Empty array if none.
`processing`	object	No	Normalization provenance.

4.1.1 Synthetic `messageId`

When the `Message-ID` header is absent or not valid (§5.1), implementations MUST still produce a non-null `messageId`. It MUST be deterministic: the same source message MUST always yield the same ID.

When the complete original message is available, the RECOMMENDED form is:

```
generated-{prefix}@aecs.local
```

where `{prefix}` is the first 32 characters of the lowercase hex SHA-256 digest of the message's original octets (the same bytes represented by `content.rawFull` when populated).

Implementations that normalize without access to full message octets MUST document their deterministic scheme. A synthetic ID MUST NOT be used when a valid `Message-ID` header is present.

4.2 `metadata`

Field	Type	Description
`metadata.from`	Address	Parsed `From` header.
`metadata.to`	Address[]	Parsed `To` header recipients.
`metadata.cc`	Address[]	Parsed `CC` header recipients.
`metadata.bcc`	Address[]	Parsed `BCC` header. Typically absent from received messages.
`metadata.subject`	string | null	Decoded `Subject` header value.
`metadata.date`	string | null	`Date` header value normalized to ISO 8601 UTC. `null` if the header is absent or unparseable — see §6.
`metadata.timestamp`	number | null	Unix epoch (seconds, UTC). Parsed from `metadata.date`; `null` under the same conditions as `metadata.date` — see §6.

Address object:

```
{ "name": "Alice Smith", "email": "alice@example.com" }
```

`name` is `null` when no display name is present.

4.3 `content`

The content object provides the same message body at six processing levels. Implementations SHOULD populate all levels they are capable of producing. Fields the implementation cannot produce MUST be `null`.

Field	Description
`content.rawFull`	Complete original RFC 5322 message — all headers, MIME parts, encodings, exactly as received. Suitable for archival and re-parsing.
`content.raw`	The latest message body only. Quoted reply history is stripped at the MIME level. Headers are excluded.
`content.html`	HTML rendition of the latest message content. `null` if the message has no HTML part.
`content.text`	Plain text rendition of the latest message content, decoded from any transfer encoding.
`content.clean`	Plain text with email signatures and quoted reply chains removed using heuristic detection. May be imperfect.
`content.forAI`	Derived from `clean`. Additionally: whitespace normalised, inline image references removed, forwarded-message headers collapsed to a single summary line. This is the field AI consumers SHOULD use as their primary input.

Consumers preferring minimal context window usage should use `content.forAI`. Consumers requiring fidelity to the original should use `content.rawFull`.

4.4 `thread`

Field	Type	Description
`thread.position`	number | null	Position of this message within the conversation, ordered by ascending `metadata.timestamp` (the `Date` header value — see note below), where `0` = earliest. `null` when the implementation cannot determine position without loading the rest of the thread (see below).
`thread.inReplyTo`	string | null	The raw value of the `In-Reply-To` header (Message-ID, angle brackets stripped).
`thread.references`	string[]	Ordered list of Message-IDs from the `References` header, earliest first.

On `thread.position`:

- `thread.position` cannot be computed from a single message — it requires knowing every other message in the thread. An implementation normalizing one message in isolation (e.g. as it arrives) MUST set `thread.position` to `null`; it MUST only be populated once the full set of messages sharing a `threadId` is available and sorted.
- The ordering key is `metadata.timestamp` (i.e. the sender-supplied `Date` header), **not** the order in which an implementation received or processed each message. These are different

orderings whenever mail is delayed, backdated, or a sender's clock is skewed — and per §7, `Date` is sender-controlled, untrusted input. Implementations that need true receipt/processing order for robustness against clock skew or spoofing SHOULD use `processing.processedAt` (§4.6) for that purpose instead of `thread.position`.

- Ties (two messages with identical `metadata.timestamp`) MAY be broken by `messageId` string comparison for a stable, deterministic sort; this spec does not mandate a specific tiebreak beyond requiring one to exist so position assignment is reproducible.

4.5 `attachments`

Each element in the `attachments` array describes one MIME attachment.

Field	Type	Description
`id`	string | null	Optional stable identifier for this attachment within the message. RECOMMENDED derivation: `` `${messageId}:${index}` `` where `index` is the attachment's 0-based position in MIME order. Implementations that don't populate this MUST use `null`, not a random value that would change between normalizations of the same message.
`filename`	string	Decoded filename from `Content-Disposition` or `Content-Type` `name` parameter.
`contentType`	string	MIME type (e.g. `application/pdf`).
`size`	number	Size in bytes.
`cid`	string | null	Content-ID for inline attachments (`Content-ID` header, angle brackets stripped). `null` for non-inline attachments.

Attachment binary content is not included in `NormalizedEmail`. Implementations store and reference it separately.

4.6 `processing`

Field	Type	Description
`processing.processedAt`	string	ISO 8601 UTC timestamp of when this normalization was produced.
`processing.specVersion`	string	The AECS version used (e.g. `"1.0"`).

---

5. Threading Algorithm

5.1 Validity of a Message-ID

A **valid Message-ID** is a value that, after trimming surrounding whitespace and stripping one optional pair of enclosing angle brackets, is non-empty and contains exactly one `@` character with a non-empty sequence of characters on each side (informally following the `msg-id` grammar of RFC 5322 §3.6.4). Implementations MAY apply the full RFC 5322 `msg-id` ABNF for stricter validation — the rule above is the minimum bar and is sufficient to satisfy this specification's determinism requirements. A header value that is absent, empty, or fails this test is **not valid** and MUST be treated as if that header were absent for the purposes of §5.2.

5.2 Algorithm

All implementations MUST calculate `threadId` using the following deterministic algorithm, evaluated in order:

1. If a `References` header is present, scan its entries in order and use the **first entry that is a valid Message-ID** (§5.1) as `threadId`. An invalid entry is skipped, not treated as ending the header — e.g. `References: garbage, <valid@example.com>` resolves to `valid@example.com`, not to rule 2.
2. Otherwise, if an `In-Reply-To` header is present and is a valid Message-ID, use it as `threadId`.
3. Otherwise, if the message's own `Message-ID` header is present and valid, use it as `threadId`.
4. Otherwise (no valid Message-ID found anywhere on the message), generate a deterministic fallback: `SHA-256(from_email + ":" + subject_lowercased_trimmed + ":" + date_utc_iso8601)`, encoded as lowercase hex. See §5.4 for the exact encoding this hash MUST use.

5.3 Requirements

- The result MUST be identical for every message in the same conversation, regardless of the order messages are processed — §5.2 depends only on one message's own headers, never on other messages that have or haven't been seen.
- Angle brackets in Message-IDs MUST be stripped before comparison or storage (`<abc@example.com>` → `abc@example.com`).
- Whitespace in Message-IDs MUST be trimmed.
- The fallback hash (rule 4) is a last resort. Implementations SHOULD log a warning when it is used.

5.4 Encoding for the Fallback Hash (Rule 4)

The fallback hash is only reproducible across independently-written implementations if every input to it is normalized identically first:

- `from_email` is the addr-spec portion of the `From` header (display name excluded). Implementations SHOULD normalize it to lowercase. A missing `From` address contributes the empty string.
- `from_email`, `subject_lowercased_trimmed`, and `date_utc_iso8601` MUST each be Unicode normalized to NFC before concatenation. Two visually identical strings can have different byte sequences (e.g. a precomposed vs. combining diacritic) — without NFC normalization, two correct implementations can hash the same logical subject to different values.
- The concatenated string MUST be UTF-8 encoded before hashing.
- `subject_lowercased_trimmed` MUST use the fully decoded Unicode subject (RFC 2047 encoded-words decoded first, if present), lowercased using the **locale-independent** Unicode default case mapping (e.g. JavaScript `String.prototype.toLowerCase()`, not `toLocaleLowerCase()` — the Turkish locale's dotless-ı mapping for `I` / `i` is a well-known source of divergence), with leading/trailing whitespace trimmed. A missing subject contributes the empty string, not the text `"null"`.
- `date_utc_iso8601` MUST be formatted exactly as `metadata.date` (§6): ISO 8601, UTC, `Z` suffix, second precision — e.g. `2026-06-29T10:00:00Z`. When `metadata.date` is `null`, `date_utc_iso8601` contributes the empty string (the separating `:` in the concatenation is still present).

5.5 Design Rationale: Static vs. Dynamic (JWZ-Style) Threading

Mail clients implementing the Jamie Zawinski ("JWZ") threading algorithm build a container tree incrementally and *re-parent* messages as earlier context arrives out of order — an orphaned reply gets retroactively attached once its missing parent finally shows up. AECS-1 deliberately does not do this: `threadId` (§5.2) is a pure function of one message's own headers and never depends on which other messages have or haven't been seen. This is a narrower guarantee than JWZ reparenting, traded for the property this spec is built around — `threadId` is computable from a single message in isolation, with no external state, and is guaranteed stable regardless of processing order (§5.3). An implementation that wants JWZ-style merge-on-discovery behavior MAY build it as an application-layer feature that groups AECS-1 `threadId`s together after the fact; that grouping logic is out of scope here.

---

6. Timestamps

- All timestamps in `NormalizedEmail` MUST be in UTC.
- `metadata.date` MUST be ISO 8601 with explicit UTC offset (`Z` or `+00:00`).
- `metadata.timestamp` MUST be Unix epoch seconds derived from `metadata.date` (integer, floor of UTC seconds — no fractional component).
- `processing.processedAt` MUST be ISO 8601 UTC.
- If the `Date` header is absent or unparseable, `metadata.date` and `metadata.timestamp` MUST be `null`. Implementations MUST NOT substitute processing time, receipt time, or any other guessed value.

6.1 `Date` Header Parsing

When normalizing from RFC 5322 messages, implementations MUST parse the `Date` header and produce `metadata.date` in the form `YYYY-MM-DDTHH:MM:SSZ` with **no fractional seconds**. Implementations MAY also accept values already in ISO 8601 form.

`metadata.timestamp` MUST be the integer Unix epoch second corresponding to that instant in UTC. Two implementations parsing the same `Date` header MUST produce identical `metadata.date` and `metadata.timestamp` values.

---

7. Security Considerations

This specification defines data structure only. It does not mandate sanitization rules, output delimiters, or content filtering.

Implementers and consumers should note:

- All email content — including `subject`, sender names, and body fields at every level — originates from an untrusted external source.
- The `forAI` field reduces noise but does not sanitize for prompt injection. An adversary can craft email content designed to manipulate an AI system that processes it as instructions.
- Safe usage of any `content.*` field with an LLM is the responsibility of the consuming application.
- Implementations are encouraged to offer an optional scanning layer and attach findings as metadata outside this core schema. This spec does not define that layer.
- `content.rawFull` in particular MUST be treated as fully untrusted input if re-parsed downstream.
- **`content.html` is live, attacker-influenced markup, not just an LLM-injection vector.** It commonly contains remote-resource references (`<img src>`, tracking pixels, remote CSS

`url()`). A consuming application that renders `content.html` directly, or that eagerly fetches URLs found in it (e.g. for a link-preview feature), is exposed to SSRF (the fetch can target internal/private network addresses reachable from the fetching service) and to the classic email tracking-pixel privacy leak (fetching the URL confirms to the sender that the message was opened, and from roughly where). Consumers rendering `content.html` SHOULD do so in a sandboxed context (e.g. a sandboxed iframe with remote image loading disabled by default) and SHOULD NOT server-side fetch URLs discovered in email content without the same allow-listing/network-egress controls used for any other untrusted-URL-fetching feature.

---

8. Versioning

This specification follows semantic versioning (`MAJOR.MINOR.PATCH`).

- Breaking changes to the `NormalizedEmail` schema increment the major version.
- Additive, non-breaking changes increment the minor version.
- The `processing.specVersion` field in each normalized object SHOULD record the major and minor version used (e.g. `"1.0"`).

Current version: **1.0.0**

Release History

Version	Date	Notes
1.0.0	2026-07-03	First stable release. Adds §4.1.1 (synthetic `messageId`), §6.1 (`Date` parsing), and clarifies §5.4 fallback-hash inputs.
1.0.0-draft	2026-06-29	Initial public draft.

---

9. Extensibility

Implementations MAY add fields to `NormalizedEmail` or any nested object provided they:

- Do not reuse names defined in this specification with a different type or semantics.
- Use a namespaced key for custom fields (e.g. `"x_myapp_score": 0.87`).

Consumers MUST ignore unknown fields to remain forward-compatible.

---

10. Conformance

Requiredness is stated throughout this document as it comes up (§2, §4, §5, §6). This section collects it into one checklist. An implementation is **AECS-1-conformant** if and only if it satisfies every point below:

1. Every produced object has non-null `messageId` and `threadId` (§4.1) — these are the only two fields this spec requires to always be present and non-null. When no valid `Message-ID` header exists, `messageId` MUST be synthetic per §4.1.1.
2. Producers SHOULD represent unpopulated optional fields as explicit `null` (or, for `attachments`, an empty array). A conformant consumer MUST treat an omitted field and an explicit `null` identically (§2, §4.1).
3. `threadId` is computed using the exact algorithm in §5.2, including the validity definition in §5.1 and the encoding rules in §5.4 for the fallback hash — not an approximation that happens to agree on common-case input.
4. `thread.position` is `null` unless computed from a fully-available, timestamp-sorted thread per §4.4 — never a value guessed from a single message.
5. All timestamps satisfy §6 exactly: UTC, ISO 8601 with explicit offset, Unix epoch seconds, and `null`-not-guessed when the source `Date` header is absent or unparseable.
6. Unknown/custom fields are namespaced per §9 when producing, and ignored (not an error, not a validation failure) when consuming.
7. `content.rawFull`, if populated, is byte-faithful to the original message — conformance does not require populating it (§2's "flexible by design"), but if present it MUST NOT be normalized, re-encoded, or otherwise altered from the source.

A conformant implementation is NOT required to populate every `content.*` level (§4.3 already says implementations SHOULD populate what they're capable of, not MUST populate all) — the bar is that *whatever* is populated follows the rules above, not that everything is populated. See `specs/conformance/` for machine-checkable fixtures covering points 3–5, and `specs/schema/normalized-email.schema.json` for a JSON Schema covering points 1, 2, and 6 (shape and nullability).

Appendix A: Example — Reply in a Thread

This example shows a reply message. Note how the content levels diverge as processing increases.

```
{
  "messageId": "reply789@mail.example.com",
  "threadId": "root456@mail.example.com",
  "metadata": {
    "from": { "name": "Bob", "email": "bob@example.com" },
    "to": [{ "name": "Alice", "email": "alice@example.com" }],
    "cc": [],
    "bcc": [],
    "subject": "Re: Project update",
    "date": "2026-06-29T14:32:00Z",
    "timestamp": 1782743520
  },
  "content": {
    "rawFull": "From: Bob <bob@example.com>\r\nTo: Alice
<alice@example.com>\r\nSubject: Re: Project update\r\nDate: Sun, 29 Jun 2026
14:32:00 +0000\r\nMessage-ID: <reply789@mail.example.com>\r\nIn-Reply-To:
<root456@mail.example.com>\r\nReferences: <root456@mail.example.com>\r\nContent-
Type: text/plain; charset=UTF-8\r\n\r\nThanks Alice, looks good to me. Let's go
ahead.\r\n\r\nOn Sun, 29 Jun 2026 at 09:00, Alice <alice@example.com> wrote:\r\n> Hi
Bob, just wanted to share the latest project update.\r\n> Everything is on track for
the Thursday deadline.\r\n>\r\n> -- \r\n> Alice Smith | Product Lead",
    "raw": "Thanks Alice, looks good to me. Let's go ahead.\r\n\r\nOn Sun, 29 Jun
2026 at 09:00, Alice <alice@example.com> wrote:\r\n> Hi Bob, just wanted to share
the latest project update.\r\n> Everything is on track for the Thursday
deadline.\r\n>\r\n> -- \r\n> Alice Smith | Product Lead",
    "html": null,
    "text": "Thanks Alice, looks good to me. Let's go ahead.\n\nOn Sun, 29 Jun 2026
at 09:00, Alice <alice@example.com> wrote:\n> Hi Bob, just wanted to share the
latest project update.\n> Everything is on track for the Thursday deadline.\n>\n> --
\n> Alice Smith | Product Lead",
    "clean": "Thanks Alice, looks good to me. Let's go ahead.",
    "forAI": "Thanks Alice, looks good to me. Let's go ahead."
  },
  "thread": {
    "position": 1,
    "inReplyTo": "root456@mail.example.com",
    "references": ["root456@mail.example.com"]
  },
  "attachments": [],
  "processing": {
    "processedAt": "2026-06-29T14:32:01Z",
    "specVersion": "1.0"
  }
}
```


`rawFull` contains the complete RFC 5322 message exactly as received. `raw` is the body only, with headers removed but quoted history still present. `text` is the same decoded to clean line endings. `clean` and `forAI` strip the quoted chain and signature, leaving only Bob's actual reply.

`thread.position` is `1` here because this example represents the message as it appears *after* thread reconciliation (this is the second of two messages, following the root shown in `references`). A single, isolated `parse()` of this message with no knowledge of the rest of the

thread would instead produce `thread.position: null` per §4.4.

Appendix B: Reference Implementation

`@mvrx/mail` is the TypeScript reference implementation of AECS-1. Core modules:

Module	Spec coverage
`parse()`	Full `NormalizedEmail` production from RFC 5322/MIME (§3–§4, §6)
`resolveThreadId()`	Threading algorithm (§5)
`normalizeDate()`	Timestamp rules (§6)
`EmailThread`	`thread.position` assignment (§4.4)

Conformance tests in `packages/mail/test/core.test.mjs` run every fixture in `specs/conformance/fixtures/`.

- GitHub: github.com/mvrxapp/mail
- npm: `@mvrx/mail`